# CREDIT CARD FRAUD DETECTION USING EXTREME GRADIENT BOOSTING

Nguyen Trong Dai  Mai Huy Dang  Vu Quoc Huy

AI66A Group 1

April 29, 2026

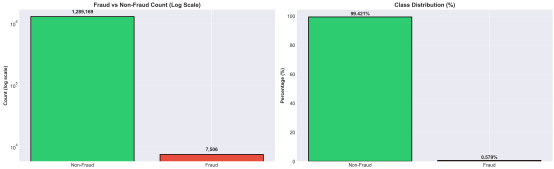
# Problem and Goal

- Fraud is rare but high-impact: only around 0.5% of transactions are fraudulent.
- A model can get very high accuracy while still missing most fraud cases.
- Main objective: maximize fraud detection while keeping false alerts low.
- Priority metrics:
  - Recall (catch fraud)
  - F2-score (recall weighted 2x more than precision)
  - PR-AUC (best metric for imbalanced classification)

# Dataset Overview

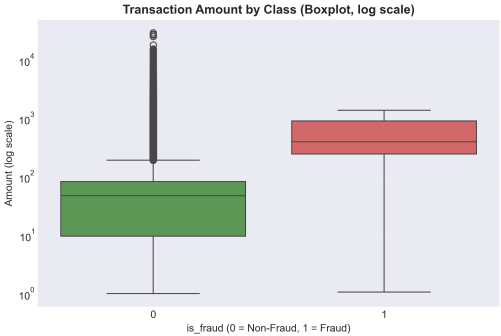
| Split | Transactions | Fraud | Rate |
|---|---|---|---|
| Train | 1,296,675 | 7,506 | 0.579% |
| Test | 555,719 | 2,145 | 0.386% |

- Source: Kaggle credit card fraud dataset.
- 23 raw columns (time, amount, location, merchant, user attributes).
- No missing values and no duplicate transactions.



# Data Quality and Leakage Control

- Outliers in `amt` were kept because they carry real fraud signal.
- `trans_date_trans_time` was dropped after extracting time features.
- Target encoding used out-of-fold estimation (StratifiedKFold, k=5).
- Unseen categories in test data are mapped to global fraud rate.
- This prevents target leakage from test into training.



# Feature Engineering

**Engineered features**

- Time: `hour`, `day_of_week`, `month`, `is_weekend`, `is_night`
- Amount transforms: `amt_log`, `amt_zscore`
- User age: `age`
- Geo distance: `distance_km`
- Frequency encodings: category, merchant, city
- Target encodings: category, merchant, city fraud rates

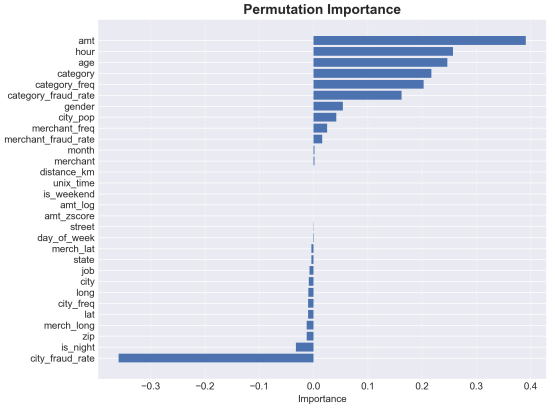
**Final 9 features used**

- amt
- hour
- age
- city_pop
- gender
- category_freq
- merchant_freq
- category fraud rate
- merchant fraud rate

**Notable finding:** dropping `city_fraud_rate` improved recall/F2 by about 30–40 points.

# Feature Selection Evidence

- SHAP explains feature contribution at transaction level.
- Permutation importance validates whether a feature helps global performance.
- Features with near-zero or negative importance were removed.
- Harmful features removed:
  - `city_fraud_rate` (around -0.36)
  - `is_night` (negative effect)



# Modeling Approach

- Evaluated 5 models on the same feature set:
  - Logistic Regression (baseline)
  - Random Forest
  - LightGBM
  - XGBoost
  - MLP (PyTorch)
- Class imbalance handled by `scale_pos_weight = 171.75`.
- XGBoost chosen for best overall trade-off among PR-AUC, recall, F2, and false alerts.
- Hyperparameter optimization: Optuna (TPE), 50 trials, objective = PR-AUC.

# Imbalance Handling Comparison

| Strategy | Precision | Recall | F2 | PR-AUC | False Alerts |
|---|---|---|---|---|---|
| `scale_pos_weight` | **65.6%** | 86.2% | **0.8115** | **0.8871** | **969** |
| Undersampling | 27.68% | 95.34% | 0.6403 | 0.8856 | 5,344 |
| SMOTE | 6.52% | **96.04%** | 0.2564 | 0.8378 | 29,536 |

- `scale_pos_weight` gave the best balance without modifying original data distribution.
- Resampling raised recall but produced too many false alerts.

# Evaluation and Threshold Strategy

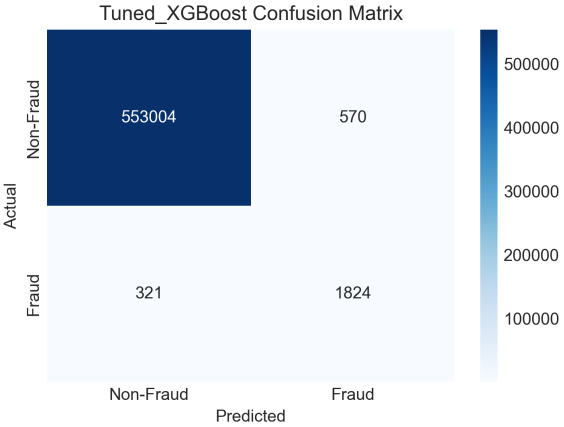
- Accuracy is excluded due to severe class imbalance.
- Core metrics:
  - Recall: minimize missed fraud (FN)
  - F2-score: optimize recall with precision control
  - PR-AUC: model selection and Optuna objective
- Threshold optimization performed on Precision-Recall curve.
- Default threshold 0.5 was replaced by F2-optimized threshold 0.9534.
- Result: precision improved strongly while recall stayed high.

# Final Model Performance

**Tuned XGBoost + threshold 0.9534**

| Metric | Value |
|---|---|
| PR-AUC | 0.8975 |
| ROC-AUC | 0.9978 |
| Precision | 76.2% |
| Recall | 85.0% |
| F2-score | 0.8311 |
| False Alert Rate | 0.10% |

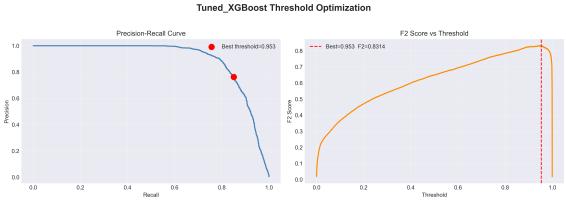
- TP: 1,824
- FP: 570
- FN: 321
- TN: 553,004



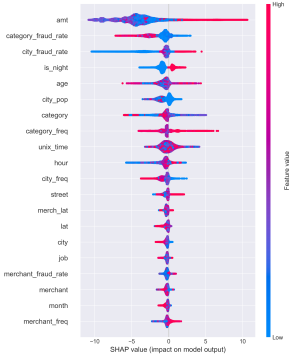
# Baseline vs Tuned XGBoost

| Metric | Baseline | Tuned |
|---|---|---|
| PR-AUC | 0.8872 | 0.8975 |
| Precision | 65.63% | 76.2% |
| Recall | 86.25% | 85.0% |
| F2-score | 0.8115 | 0.8311 |
| False Alert Rate | 0.175% | 0.10% |

- Precision: +10.57 points
- F2-score: +0.0196
- False alert rate: down by 0.075 points



# Model Explainability




**Main insights**

- `amt` is the strongest fraud signal.
- Category fraud rate and `hour` are key drivers.
- SHAP + permutation helped remove harmful features.

# Deployment Plan

**Saved artifacts**

- `models/xgb_fraud_final.pkl`
- `models/model_metadata.json` (feature order, threshold, metrics, params)

**Inference contract**

- Input must contain the 9 engineered features in correct order.
- Output includes `fraud_probability` and `is_fraud_predicted`.

**Path to production**

- REST API (FastAPI/Flask)
- Feature store for live encodings
- Model registry and versioning
- Drift monitoring and scheduled retraining

# Conclusion

- XGBoost is the best model for this tabular fraud detection task.
- Final system catches 85% fraud with 76.2% precision and only 0.10% false alerts.
- Strongest gains came from:
  - robust feature selection
  - proper imbalance handling
  - F2-based threshold optimization
- This pipeline is a solid baseline for real-time fraud detection deployment.